

14 — Una lingua che manca

La domanda a cui risponde: la fonte dei dati non ha quello che ti serve, ma ce l'ha in un'altra forma. Prendere quella forma è un compromesso o un errore? E quando la prendi, **quale parte** dei dati va tradotta e quale va lasciata stare?

Il fatto

Cinque carte vere, in mano, tutte italiane:

carta	set	esce nell'app?
Salamence 19/97	EX Drago	no
Laios ex 94/97	EX Drago	no
Jirachi 8/101	EX Leggende Nascoste	no
Spinda 48/101	EX Leggende Nascoste	no
Articuno ex 032	Nintendo Black Star Promos	no

Il progetto ha scelto TCGdex **in italiano** per una ragione precisa (vedi CLAUDE.md): la collezione fisica è italiana, e un'app che mostra "Iron Crown" con la scansione inglese non ti fa ritrovare la carta nel mazzetto. Fallisce proprio nel suo scopo.

Solo che TCGdex tiene una scheda **per lingua**, e per 79 set la lista italiana è vuota:

```
GET /v2/it/sets/ex3 → {"name": "EX Drago", "cardCount": {"official": 97}, "cards": []}
GET /v2/en/sets/ex3 → {"name": "Dragon", "cardCount": {"official": 97}, "cards": [100 carte]}
```

Nome del set tradotto, conteggio giusto, **zero carte**. EX Drago in Italia è uscito davvero: manca il dato, non la stampa.

Il downloader scartava quei set di proposito, con un commento che diceva "set mai usciti in italiano" — una diagnosi sbagliata su un sintomo giusto. Sono 5.910 carte, il 28% di tutto il catalogo, e sono le più vecchie: esattamente quelle che stanno nei raccoglitori di chi gioca da vent'anni.

La domanda vera

"Ripiegare sull'inglese" suona come una resa. Non lo è, se ci si accorge che i dati di una carta **non sono tutti della stessa natura**. Provate a guardarli chiedendovi *a cosa serve questo campo*:

campo	valore inglese	a cosa serve	è testo?
category	Pokemon	il motore filtra i Pokémon	no, è una chiave
stage	Basic	costruire le linee evolutive	no, è una chiave
types	Fire	contare le energie per tipo	no, è una chiave
attacks[].cost	['Fire', 'Colorless']	calcolare il fabbisogno di energie	no, è una chiave
rarity	Rare Holo	il filtro per rarità	no, è una chiave
name	Laios ex	leggerlo	sì
attacks[].name	Luster Purge	leggerlo	sì

Le prime cinque righe **sembrano** testo — sono parole, hanno le maiuscole — ma nel codice sono etichette di un insieme chiuso, e vengono confrontate:

```
// src/engine/analisi.js
if (voce.carta?.categoria !== 'Pokémon') continue;
```

Una carta che arriva con categoria: 'Pokemon' (senza accento) non fa scattare nessun errore. **Sparisce.** Non compare nei conteggi, non entra in nessun mazzo, non viene segnalata da niente. È il tipo di guasto peggiore: silenzioso e plausibile.

Le ultime due righe invece sono davvero testo, e per quelle una traduzione non esiste: inventare “Purificazione Lucente” vorrebbe dire **scrivere sulla carta cose che sulla carta non ci sono.**

Da qui la regola che governa tutto il ripiego:

Si traduce il vocabolario chiuso, si lascia stare il testo libero.

E il vocabolario chiuso non è un'ipotesi: TCGdex lo pubblica, in entrambe le lingue.

```
curl https://api.tcgdex.net/v2/en/types # ["Colorless","Darkness",...]
curl https://api.tcgdex.net/v2/it/types # ["Acqua","Drago",...]
```

Il codice

In tools/scarica-set.mjs, il ripiego è tre righe:

```
let elenco = dettaglio.cards ?? [];
let lingua = 'it';
let fonte = API;
if (elenco.length === 0) {
  const ripiego = await prendiJson(`${API_RIPIEGO}/sets/${set.id}`);
  elenco = ripiego.cards ?? [];
  lingua = 'en';
  fonte = API_RIPIEGO;
}
```

Notare la condizione: **zero assoluto.** Un set con *qualche* carta italiana si tiene com'è. Rattoppare i buchi darebbe un set mezzo e mezzo, in cui non si sa più cosa si sta guardando — e mescolare è proprio ciò che si vuole evitare.

La traduzione è una tabella e una funzione che non fa nulla se non trova:

```
function traduci(campo, valore) {
  if (valore == null) return valore;
  return VOCABOLARIO[campo][valore] ?? valore;
}
```

Il ?? valore è la scelta di progetto più densa del file. Significa: *ciò che non so tradurre lo lascio passare com'è*, invece di buttarlo o di inventare. E si applica sempre, anche ai dati italiani, dove semplicemente non trova niente da cambiare — un ramo if (lingua === 'en') in meno da sbagliare.

Il costo dei nomi che non hanno gemello

Alcune cose in italiano **non esistono e basta**, perché quelle ere qui non sono mai uscite:

- LEVEL-UP, gli LV.X di Diamante & Perla;
- le rarità Rare PRIME e LEGEND di HeartGold & SoulSilver.

Per lo stadio la scelta è stata **non tradurre.** Sembra una resa, è prudenza: linee.js scarta gli stadi che non conosce e analisi.js lo dichiara con l'avviso stadio-ignoto. Inventare “Livello X” lo avrebbe fatto assomigliare a “Livello 1”, e il motore avrebbe provato a incastrarlo in una piramide evolutiva dove non sta.

Meglio un buco dichiarato di un dato plausibile e falso.

Per le rarità invece è stata aggiunta una regola in src/data/rarita.js, perché lì c'era un test che non lasciava scampo:

```
test('ogni rarità dei dati reali trova una classe: nessuna finisce in "altra"', ...)
```

Quel test legge **i dati veri**, non delle fixture, e infatti è caduto appena i set inglesi sono entrati. Era il suo mestiere.

Il campo che dice la verità

I set che vengono dal ripiego sono marcati nell'indice:

```
{ "id": "ex3", "nome": "EX Drago", "totale": 97, "lingua": "en", ... }
```

Il campo si scrive **solo** quando vale `en`: assente significa italiano. Non è pigrizia, è che così i 110 set già presenti non cambiano di una virgola e il diff mostra soltanto ciò che è nuovo.

Serve perché l'app deve poterlo **dire**. Senza, un utente legge “Luster Purge” e crede che sulla sua carta ci sia scritto così. Il dato mescolato in silenzio è il vero danno: non l'inglese in sé, ma l'inglese spacciato per italiano.

Il campo arriva a schermo come una pastiglia **EN** (`src/ui/lingua-set.js`) in tre punti: la riga dei candidati quando si aggiunge, la card nella griglia, e la barra del visore a schermo intero. Tre host che non si conoscono fra loro — ed è proprio il motivo per cui la pastiglia è un modulo e non tre pezzi di HTML copiati: scritta tre volte, tre volte sarebbe diventata diversa.

Un dettaglio che si scopre solo collegandola: la griglia passa al visore una **proiezione** della voce (`card._voce`), non la voce intera. La lingua andava aggiunta lì a mano, o si fermava per strada — proprio prima del punto in cui serve di più, perché nel visore sotto la carta non c'è nessun altro testo e la scansione è tutto ciò che si legge.

Le crepe che si scoprono solo facendolo

Due cose sono emerse solo lanciando il download davvero, ed entrambe valgono più del codice che le risolve.

1. Una carta ha ucciso ottanta set. Il set `exu` contiene i 28 Unown, che invece dei numeri usano le lettere: A, B, ... Z, !, e ?. L'id di quest'ultima è `exu-?`, e l'API non riesce a servirla nemmeno con il punto interrogativo codificato:

```
GET /v2/en/cards/exu-%3F → 404
```

Il download si è fermato lì, dopo venti minuti di lavoro. La lezione non è “gestire i 404”: è che in un lavoro lungo e ripetitivo **il fallimento di un elemento non deve poter fermare l'insieme** — ma non deve nemmeno sparire.

```
try {
  completa = await prendiJson(`${fonte}/cards/${breve.id}`);
} catch {
  perse.push(breve.localId ?? breve.id);
}
```

Le carte perse si contano e si stampano. Una su 5.910, dichiarata.

2. L'indice si riscrive da capo. `scarica-set.mjs` sovrascrive `indice.json`, e `serie` e `uscita` — che ci mettono altri due strumenti — spariscono. Non è un bug: è una **catena di strumenti** con un ordine obbligatorio, e la catena è documentata in `CLAUDE.md` proprio perché il codice da solo non la impone.

Chi viene da Java riconoscerà la differenza fra un `mvn package` che sa da sé cosa dipende da cosa, e uno script che si fida di te. Qui non c'è build system: l'ordine sta nella documentazione e nella testa di chi lancia. Che è un costo reale del vincolo “zero build” — un costo scelto, non subito.

Cosa NON ha risolto

Articuno `ex` continua a non entrare, ed è istruttivo capire perché: è una **promo**, e sulla carta c'è stampato 032 senza nessun `/xxx`. La maschera di inserimento pretende il totale:

```
if (!numero || !totale) return;
```

I dati adesso ci sono, la strada per arrivarci no. Sono due guasti distinti che sembravano uno, e li separa una domanda sola: *questa carta manca dai dati, o manca il modo di chiederla?*

Vale per oltre 750 promo **italiane** già presenti nel repository, che oggi non sono catalogabili per lo stesso motivo. Il seguito naturale è un indice dei nomi (nome + numero identifica il 97% delle carte; numero + totale molto meno).

Verifica

1. `traduci` lascia passare i valori che non conosce invece di buttarli o sostituirli con un default. Elenca due modi in cui il comportamento opposto — scartare l'ignoto — avrebbe reso i dati **peggiori** invece che più puliti.
2. La categoria `Pokemon` senza accento non provoca un errore: fa sparire la carta dai conteggi. Progetta un test che avrebbe intercettato il problema *prima* di scaricare 5.910 carte. Su cosa lo faresti girare, dati veri o fixture, e perché?
3. Il campo `lingua` viene scritto solo quando vale `'en'`. Chi legge deve quindi trattare l'assenza come `'it'`. Quando è una buona idea, e quando invece un default implicito diventa una trappola?
4. Per `LEVEL-UP` si è scelto di **non** tradurre, per le rarità inglesi sì. Qual è la differenza fra i due casi? (Suggerimento: cosa fa il codice con un valore che non riconosce, in un caso e nell'altro?)